LAPORAN PRAKTIKUM

STRUKTUR DATA

PEKAN KE 4

Disusun Oleh :

Muhammad Irfan 2511531008

Dosen Pengampu : Dr Wahyudi S.T M.T

Asisten Praktikum : Muhammad Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2025

KATA PENGANTAR

Puji syukur atas kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum dari mata kuliah Struktur Data dengan judul Queue dengan baik dan benar.

Laporan ini disusun sebagai bentuk hasil kegiatan praktikum Struktur Data yang di mana membahas tentang bahasa pemrograman java dan penggunaan Queue serta kegunaan masing-masing percabangan tersebut serta cara penerapannya.

Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna baik dari segi penulisannya dan dari segi pembahasannya. Oleh karena itu saya ingin mengucapkan terima kasih kepada dosen pengampu, asisten praktikum yang telah membimbing serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, penulis ucapkan terima kasih pada dosen pengampu yang telah memberikan arahan dan semoga laporan ini dapat bermanfaat baik bagi penulis maupun pembaca dalam pemahaman tentang Queue.

DAFTAR ISI

DAFTAR ISI 2

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB II PEMBAHASAN 2

1.Class QueueArray_2511531008 2

Kode program 2

2. Class QueueArrayDriver_2511531008 4

Kode program 4

Ouput 5

3.Class ReverseData_2511531008 5

Kode program 5

Ouput 6

4. QueueLinkedList_2511531008 6

kodeprogram 6

Output 6

5.IterasiQueue_2511531008 7

Ouput 7

BAB III PENUTUP 8

3.1 Kesimpulan 8

TUGAS PEKAN 4 9

1.Class AntrianLocket_2511531008 9

2. Class Queue_251153108 11

BAB I
PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen yang paling penting untuk menyimpan nilai dan mengelola nilai secara baik dan efisien. Salah satu struktur data yang sering di pakai adalah Queue yang menerapkan prinsip FIFO Firts in First out, dimana data pertama yang masuk menjadi data yang pertama keluar. Konsep ini banyak ditemui dalam kehidupan sehari hari, seperti antrian di kasir,buat schedulling dan lain lain

Dalam Praktikum ini, pemahaman tentang queue sangat penting untuk melatih berpikir secara logika pemrograman. Implementasi Queue dapat menggunakan array maupun linked list. Pada Praktikum ini, diharapkan mahasiswa mampu memahami, menganalisis serta mengimplementasikan dalam kode program bahasa java

1.2 Tujuan Praktikum

Tujuan utama dari praktikum ini adalah memberikan pemahaman kepada mahasiswa tentang Konsep Queue serta implementasi di dalam pemrograman bahasa Java. Selain itu bagaimana pengaplikasian berbagai berbagai-macam problem dengan menggunakan Queue

1.3 Manfaat Praktikum

Manfaat utama dari praktikum ini adalah mahasiswa diharapkan dapat memahami konsep, operasi operasi dan mengimplementasikan dalam bahasa java. Selain itu, mahasiswa juga diharapkan untuk dapat mengolah dan meningkatkan analisis terhadap problem dengan menggunakan Queue.

BAB II
PEMBAHASAN

1.Class QueueArray_2511531008

Kode program

```java
package pekan4_2511531008;
import java.util.*;

public class QueueArray_2511531008 {
    int front_1008,rear_1008,size_1008;
    int capacity_1008;
    int array_1008[];
    public QueueArray_2511531008(int capacity_1008){
        this.capacity_1008 = capacity_1008;
        front_1008 = this.size_1008 = 0;
        rear_1008 = capacity_1008 - 1;
        array_1008 = new int[this.capacity_1008];

    }
    boolean isFull_2511531008 (QueueArray_2511531008 queue) {
        return (queue.size_1008 == queue.capacity_1008);
    }
    boolean isEmpty_2511531008(QueueArray_2511531008 queue) {
        return (queue.size_1008 == 0);
    }
```

Penjelasan singkat:

- Deklarasi tipe data front,rear,size, capacity dan juga array
- Membuat objek yang bernama QueueArray_2511531008 yang atributenya terdiri dari capacity,forn,rear dan array
- Membuat method isEmpty_2511531008 yang mengecek apakah queue mempunyai nilai atau tidak. Jika kosong maka bernilai true, jika berisi maka bernilai false
- Membuat method isFull_2511531008 bernilai true jika elemen pada Queue mencapai batas maksimalnya

```java
    void enqueue_2511531008(int item_1008){
        if(isFull_2511531008(this)) return;
        this.rear_1008 = (this.rear_1008 + 1) % this.capacity_1008;
        this.array_1008[this.rear_1008] = item_1008;
        this.size_1008 = this.size_1008 + 1;
        System.out.println(item_1008 + " enqueued to queue");
    }
    int dequeue_2511531008(){
        if(isEmpty_2511531008(this)) {
            return Integer.MIN_VALUE;
        }
        int item_1008 = this.array_1008[this.front_1008];
        this.front_1008 = (this.front_1008 + 1) % this.capacity_1008;
        this.size_1008 = this.size_1008 - 1;
        return item_1008;
    }
    int front_2511531008() {
        if(isEmpty_2511531008(this))
            return Integer.MIN_VALUE;
        return this.array_1008[this.front_1008];
    }

```

Penjelasan singkat :

- Fungsi dequeue berguna untuk mengambil dan menghapus elemen dari queue
  Jika dequeue bernilai kosong maka kembalikan nilai integer.MIN_VALUE.
  Jika tidak maka nilai item di ambil dari elemen terdepan, front ditambah satu dan size dikurangi 1
  Kemudian return nilai item_1008
- Fungsi front berguna untuk melihat elemen terdepan dari queue, jika kosong maka kembalikan nilai terkecil dari integer.MIN_VALUE, jika tidak kembalikan nilai terdepan dari queue
- Fungsi enqueue berguna untuk menambahkan elemen terdepan pada sebuah queue.
  Jika queue penuh , maka tidak ada kembalian alias sudah penuh
  Jika tidak maka rear_1008 ditambah satu, menyimpan item_1008 ke posisi rear_1008 dan size ditambah 1

```java
int rear_2511531008() {
    if(isEmpty_2511531008(this))
        return Integer.MIN_VALUE;
    return this.array_1008[this.rear_1008];
}

void display_2511531008(){
    int i_1008;
    if(front_1008 == rear_1008) {
        System.out.print("\n Antrian Kosong \n");
        return;
    }

    //kunjugi dari belakang secara acak
    for(i_1008 = front_1008 ;i_1008 < rear_1008; i_1008++) {
        System.out.printf(" %d <-- ", array_1008[i_1008]);
    }
    return;
}
```

Penjelasan singkat :

- Terdapat method display yang berfungsi untuk menampilkan elemen-elemen yang terdapat pada queue
  Jika front_1008 == rear_1008 maka cetak antrian kosong
  Jila tidak, maka cetak elemen satu persatu dengan for looping
- Method rear ebrguna untuk mengambil elemen queue paling belakang

2. Class QueueArrayDriver_2511531008

Kode program

```java
package pekan4_2511531008;

public class QueueArrayDriver_2511531008 {

    public static void main(String[] args) {
        QueueArray_2511531008 queue = new QueueArray_2511531008(1000);
        queue.enqueue_2511531008(10);
        queue.enqueue_2511531008(20);
        queue.enqueue_2511531008(30);
        queue.enqueue_2511531008(40);
        System.out.println("Item di depan " + queue.front_2511531008());
        System.out.println("Item paling belakang "+ queue.rear_2511531008());
        System.out.println("tampilan queue");
        queue.display_2511531008();

        System.out.println();
        System.out.println(queue.dequeue_2511531008()+" dihapus dari queue");
        System.out.println("item di depan:" + queue.front_2511531008());
        System.out.println("item di belakang:" +queue.rear_2511531008());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue.display_2511531008();
    }
```

Penjelasan singkat :

- Membuat objek queuearray dengan nama queue
- Kemudian menambahkan elemen 10,20,30 dan 40 secara FIFO
- Mencetak elemen terdepan, belakang serta tampilan semua elemen secara FIFO
- berikutnya, 1 elemen dari depan dibuang, kemudian menampilkan elemen terdepan,belakang serta tampilan semua elemen

Ouput

```
<terminated> QueueArrayDriver_2511531008 [Java Applica
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
item di depan:20
item di belakang:40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

3.Class ReverseData_2511531008

Kode program

```
4  public class ReverseData_2511531008 {
5      public static void main(String[] args) {
6          Queue<Integer> q_2511531008 = new LinkedList<Integer>();
7          q_2511531008.add(1);
8          q_2511531008.add(2);
9          q_2511531008.add(3); //[1,2,3]
10         System.out.println("Sebelum reverse" + q_2511531008);
11         Stack<Integer> s_2511531008 = new Stack<Integer>();
12         while(!q_2511531008.isEmpty()){
13             s_2511531008.push(q_2511531008.remove());
14         }
15
16         while(!s_2511531008.isEmpty()) {
17             q_2511531008.add(s_2511531008.pop());
18         }
19         System.out.println("sesudah reverse " + q_2511531008);
20         //[3, 2, 1]
21     }
```

Penjelasan singkat:

- Membuat objek queue dengan nama q_2511531008
- Menambahkan nilai 1,2,3 ke dalam antrean queue
- Baris 10 mencetak semua elemen yang terdapat pada queue
- Baris 11, membuat objek stack dengan nama s_2511531008
- Baris 12-14 berisi perulangan while, while akan terus berulang selama q_2511531008 tidak kosong. Statement dari while berupa menpush/memasukkan nilai s_2511531008 dengan cara menghapus queue
- Baris 15-18 berisi perulangan wh9le, while akan terus berulamg selama s tidak kosong. Statrement dari while berupa memasukkan nilai stack ke dalam queue dengan cara pop (mengambil nilai stack tanpa kembalian)

Ouput

```
Console
<terminated> ReverseData_2511531
Sebelum reverse[1, 2, 3]
sesudah reverse [3, 2, 1]
```

4. QueueLinkedList_2511531008

kodeprogram

```
3  public class QueueLinkedList_2511531008 {
4
5      public static void main(String[] args) {
6          Queue<Integer> q_2511531008 = new LinkedList<>();
7          for(int i_1008 = 0; i_1008 < 6; i_1008++) {
8              q_2511531008.add(i_1008);
9          }
10         //Menampilkan isi antrian
11         System.out.println("elemen antrian " + q_2511531008);
12         //untukmenghapus kepala antrian
13         int hapus_1008 = q_2511531008.remove();
14         System.out.println("Hapus elemen = " + hapus_1008);
15         System.out.println(q_2511531008);
16         //untuk melihat antrian terdepan
17         int depan_1008 = q_2511531008.peek();
18         System.out.println("Kepala Antrian = "+ depan_1008);
19         int banyak_1008 = q_2511531008.size();
20         System.out.println("Size Antrian = " + banyak_1008);
21     }
22
```

Penjelasan singkat:

- Membuat objek queue dengan nama q_2511531008
- Menambahkan elemen queue dengan for looping, nilai nilai elemen berisi iterasi for loop dari 0 hingga 5 [0,1,2,3,4,5]
- Baris 11 mencetak nilai semua elemen pada q_2511531008
- Int hapus didapatkan dari elemen q_2511531008 yang dihapus
- Baris 15 mencetak semua elemen q_2511531008 yang elemen depan sudah dihapus
- Nilai int depan didapatkan dari elemen terdepan/antrean terdepan pada queue
- queue.size() berfungsi untuk menghitung banyak elemen pada queue

Output

```
<terminated> QueueLinkedList_2511531008 [Jav
elemen antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

5.IterasiQueue_2511531008

```
3  public class IterasiQueue_2511531008 {
4
5      public static void main(String[] args) {
6          Queue<String> q_2511531008 = new LinkedList<>();
7          q_2511531008.add("Praktikum");
8          q_2511531008.add("Struktur");
9          q_2511531008.add("Data");
10         q_2511531008.add("Dan");
11         q_2511531008.add("Algoritma");
12         Iterator<String> iterator = q_2511531008.iterator();
13         while (iterator.hasNext()) {
14             System.out.print(iterator.next() + " ");
15         }
16
17     }
18
```

Penjelasan singkat:

- Membuat objek queue dengan nama q_2511531008. Queue ini mengimplementasikan linkedlist
- Baris 7 -11 menambahkan data
- Baris 12 membuat objek yang bernama iterator. Iterator berfungsi sebagai membaca isi queue satu persatu.
- While (iterator.hasnext) akan terus berulang selagi masih data data berikutnya. Statement while berupa mencetak menampilkan nilai iterator next dan ambil data berikutnya

Ouput

```
<terminated> IterasiQueue_2511531008 [Java Applic
Praktikum Struktur Data Dan Algoritma
```

BAB III
PENUTUP

3.1 Kesimpulan

Dari praktikum ini bisa dilihat bahwa dalam Queue adalah stuktur data linear yang bekerja pada prinsip FIFO(First in first out) elemen pertama dimasukan menjadi elemen pertama di keluarkan. Pada prakitkum ini mahasiswa dapat mengerti dan dapat menggunakan operasi operasi pada Queue seperti add, remove peek dan isEmpty. Pada praktikum ini membantu mahasiswa dalam memahami konsep queue serta mengimplementasikan ke dalam kehidupan sehari hari

TUGAS PEKAN 4

1.Class AntrianLocket_2511531008

```java
package pekan4_2511531008;
import java.util.*;
public class AntrianLocket_2511531008 {
    int front_1008,rear_1008,max_1008,size_1008;
    String array_1008[];
    public AntrianLocket_2511531008 (int max_1008) {
        this.max_1008 = max_1008;
        front_1008 = this.size_1008 = 0;
        rear_1008 = max_1008 - 1;
        array_1008 = new String[this.max_1008];
    }
    boolean isFull_2511531008(AntrianLocket_2511531008 queue) {
        return(queue.size_1008 == queue.max_1008);
    }
    boolean isEmpty_2511531008(AntrianLocket_2511531008 queue) {
        return(queue.size_1008 == 0);
    }
```

```java
    void enqueue_2511531008(String data) {
        if(isFull_2511531008(this)) {
            System.out.print("antrian penuh");
        }else {
            this.rear_1008 = (this.rear_1008 + 1) % this.max_1008;
            this.array_1008[this.rear_1008] = data;
            this.size_1008 = this.size_1008 + 1;
            System.out.println(data + " enqueued to queue");
        }
    }
    String dequeue_2511531008(){
        if(isEmpty_2511531008(this)) {
            return "\n tidak ada antrean yang bisa dihapus";
        }
        String item_1008 = this.array_1008[this.front_1008];
        this.front_1008 = (this.front_1008 + 1) % this.max_1008;
        this.size_1008 = this.size_1008 - 1;
        System.out.println("antrean berhasil dihapus");
        return item_1008;
    }
```

```java
    String front_2511531008() {
        if(isEmpty_2511531008(this))
            return "kosong";
        return this.array_1008[this.front_1008];
    }

    String rear_2511531008() {
        if(isEmpty_2511531008(this))
            return "kosong";
        return this.array_1008[this.rear_1008];
    }


    void display_2511531008(){
        if(front_1008 == rear_1008) {
            System.out.print("\n Antrian Kosong \n");
            return;
        }

        //kunjugi dari belakang secara acak
        System.out.println("Isi antrian:");
        int no = 1;

        for (int i = front_1008; i <= rear_1008; i++) {
            System.out.println(no + ". " + array_1008[i]);
            no++;
        }
        return;
    }

    public void reverse_2511531008() {
        if (isEmpty_2511531008(this)) {
            System.out.println("Antrian kosong");
            return;
        }

        int start = front_1008;
        int end = rear_1008;

        for (int i = 0; i < size_1008 / 2; i++) {
            String temp = array_1008[start];
            array_1008[start] = array_1008[end];
            array_1008[end] = temp;

            start = (start + 1) % max_1008;

            end = (end - 1 + max_1008) % max_1008;
        }

        System.out.println("Antrian berhasil dibalik");
    }
```

Penjelasan singkat:

Membuat class dengan nama AntrianLocket

Membuat konstuktor dengan atribute terdiri dari , array, rear,front , dan max

Terdapat method method

Enqueue yakni menambahkan elemen sesuai antrean

Dequeue yaitu menghapus elemen antrian depan dari array

Display yakni menampilkan elemen queue secara keseluruhan

Isempty merupakan method untuk mengecek apakah queue punya elemen atau tidak, begitu juga dengan method IsFull yang mengecek apakah queue penuh atau tidak

Method reverse berfungsi untuk membalikkan antrian

2. Class Queue_251153108

```java
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        AntrianLocket_2511531008 antrian = new AntrianLocket_2511531008(10);

        int chose;
        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            chose = input.nextInt();
            input.nextLine();
```

```java
            if(chose == 1) {
                System.out.print("Masukkan nama pelanggan: ");
                String nama = input.nextLine();
                antrian.enqueue_2511531008(nama);
            }else if(chose == 2) {
                antrian.dequeue_2511531008();
            }else if(chose == 3) {
                antrian.display_2511531008();
            }else if(chose == 4) {
                antrian.reverse_2511531008();
            }else if(chose == 5) {
                System.out.println("program selesai");
            }else {
                System.out.println("list tidak valid");
            }
        }while(chose != 5);

    }
```

Ouput

Contoh ouput 1

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: wawa
wawa enqueued to queue

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
antrean berhasil dihapus

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3

 Antrian Kosong
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian kosong

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
program selesai
```

Contoh ouput 2

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: wawa
wawa enqueued to queue

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: wuwu
wuwu enqueued to queue

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: wewe
wewe enqueued to queue
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. wawa
2. wuwu
3. wewe

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu:
```